Ex 6 – Semantic Movie Recommendation System using Vector Database

Learning Objective

Implement a scalable semantic search engine that utilizes high-dimensional vector embeddings and a vector database (**FAISS**) to retrieve contextually relevant media from a large-scale dataset.

Question

Consider you are a Lead Data Engineer at CineMatch, a rapidly growing streaming platform. The current search system uses **Keyword Matching**, which means if a user searches for *"movies about space exploration,"* they only get results if the exact words like *"space"* or *"exploration"* appear in the title.

The Marketing Team reports that users are frustrated because they cannot search based on **themes, emotions, or concepts**. For example:

- Searching *"heartwarming stories"* does not return movies like *Toy Story*
- Searching *"AI and robots"* may miss relevant movies without those exact keywords

Your objective is to design a **Semantic Search Prototype** using the MovieLens dataset that understands **meaning instead of exact words**.

Your Solution Must

- Convert movie titles and genres into **high-dimensional vector embeddings** so that semantically similar words (e.g., *Galaxy* and *Space*) are close in vector space
- Use **FAISS** for efficient similarity search across large datasets
- Ensure fast retrieval even when the dataset scales to thousands or millions of movies
- Combine multiple features (Title + Genres) into a single representation for better context understanding
- Use cosine similarity (or equivalent) to measure closeness between movies and user queries

Tasks to Perform

1. Install required libraries and set up the environment for embedding generation and similarity search
2. Download the MovieLens dataset and load the movie data into a structured format
3. Clean and preprocess the dataset by handling missing values and formatting text fields
4. Combine relevant features such as **Title and Genres** into a single searchable text representation

5. Generate high-dimensional embeddings for each movie using a pre-trained embedding model
6. Store all embeddings in a **FAISS** index for efficient similarity search
7. Maintain metadata (movie ID, title, genres) linked to each embedding
8. Accept a natural language query from the user (e.g., *"a movie about friendship and adventure"*)
9. Convert the user query into an embedding using the same model
10. Perform similarity search in FAISS to retrieve the top relevant movies
11. Extract and display the matched movie titles along with similarity scores
12. Evaluate the system by comparing keyword-based results vs semantic search results

Program Code:

```
import pandas as pd
import numpy as np
import faiss
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity


movies = pd.read_csv("ml-latest-small/movies.csv")
movies.head()


# Check missing values
print(movies.isnull().sum())


# Fill missing (if any)
movies['title'] = movies['title'].fillna("")
movies['genres'] = movies['genres'].fillna("")


# Clean genres (replace | with space)
movies['genres'] = movies['genres'].apply(lambda x: x.replace('|', ' '))
```

```
movies['combined'] = movies['title'] + " " + movies['genres']
movies['combined'].head()

model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(movies['combined'].tolist(), show_progress_bar=True)
embeddings = np.array(embeddings).astype('float32')
print("Embedding shape:", embeddings.shape)

dimension = embeddings.shape[1]
index = faiss.IndexFlatL2(dimension)
index.add(embeddings)
print("Total vectors in FAISS index:", index.ntotal)
metadata = movies[['movieId', 'title', 'genres']].reset_index(drop=True)

query = input("Enter your movie query: ")
query_embedding = model.encode([query]).astype('float32')
k = 5  # top results
distances, indices = index.search(query_embedding, k)

print("\nTop Recommendations:\n")
for i, idx in enumerate(indices[0]):
    print(f"{i+1}. {metadata.iloc[idx]['title']} ({metadata.iloc[idx]['genres']})")
    print(f"   Similarity Score (distance): {distances[0][i]:.4f}\n")

def keyword_search(query, df, top_k=5):
    results = df[df['combined'].str.contains(query, case=False, na=False)]
    return results.head(top_k)
```

```
print("\n 🔹 Keyword-Based Results:\n")
kw_results = keyword_search(query, movies)

for i, row in kw_results.iterrows():
    print(f"- {row['title']} ({row['genres']})")

print("\n 🔹 Semantic (FAISS) Results:\n")
for i, idx in enumerate(indices[0]):
    print(f"- {metadata.iloc[idx]['title']} ({metadata.iloc[idx]['genres']})")
```

Output:

	movieId	title	genres
0	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy
1	2	Jumanji (1995)	Adventure|Children|Fantasy
2	3	Grumpier Old Men (1995)	Comedy|Romance
3	4	Waiting to Exhale (1995)	Comedy|Drama|Romance
4	5	Father of the Bride Part II (1995)	Comedy

```
movieId    0
title      0
genres     0
dtype: int64
```

	combined
0	Toy Story (1995) Adventure Animation Children ...
1	Jumanji (1995) Adventure Children Fantasy
2	Grumpier Old Men (1995) Comedy Romance
3	Waiting to Exhale (1995) Comedy Drama Romance
4	Father of the Bride Part II (1995) Comedy

dtype: object

```
Total vectors in FAISS index: 9742
```

```
Enter your movie query: a science fiction space adventure
```

```
Top Recommendations:

1. The Space Between Us (2016) (Adventure Sci-Fi)
   Similarity Score (distance): 0.4977

2. SpaceCamp (1986) (Adventure Sci-Fi)
   Similarity Score (distance): 0.6632

3. Journey to the Center of the Earth (2008) (Action Adventure Sci-Fi)
   Similarity Score (distance): 0.6664

4. Lost in Space (1998) (Action Adventure Sci-Fi)
   Similarity Score (distance): 0.6817

5. My Science Project (1985) (Adventure Sci-Fi)
   Similarity Score (distance): 0.6867
```

```
🔹 Keyword-Based Results:


🔹 Semantic (FAISS) Results:

- The Space Between Us (2016) (Adventure Sci-Fi)
- SpaceCamp (1986) (Adventure Sci-Fi)
- Journey to the Center of the Earth (2008) (Action Adventure Sci-Fi)
- Lost in Space (1998) (Action Adventure Sci-Fi)
- My Science Project (1985) (Adventure Sci-Fi)
```

Learning Outcome:

Upon completion of this program, students will be able to:

- Understand the limitations of traditional keyword-based search systems
- Transform unstructured text into **high-dimensional vector embeddings**
- Use **FAISS** to perform fast similarity-based retrieval
- Design a **semantic search pipeline** that captures meaning and context
- Apply vector search techniques to real-world applications like recommendation systems and search engines